

PATRÓN DE DISEÑO: FACHADA (FAÇADE)

Capítulo 22 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El patrón Fachada opera sobre un subsistema de clases. Un subsistema es un conjunto de clases que trabajan conjuntamente con el propósito de ofrecer una serie de funcionalidades relacionadas. Por ejemplo, las clases Cuenta, Dirección y TarjetaCredito que trabajan en conjunto, como parte de un subsistema, para gestionar las funcionalidades de un cliente en línea.

En aplicaciones del mundo real, un subsistema puede estar compuesto por un gran número de clases. Los clientes de un subsistema pueden necesitar interactuar con varias de sus clases para satisfacer sus necesidades. Este tipo de interacción directa entre los clientes y las clases del subsistema genera un alto grado de acoplamiento entre los objetos cliente y el subsistema (Figura 1). Cada vez que una clase del subsistema sufre un cambio, como una modificación en su interfaz, todas las clases cliente dependientes se ven afectadas.

El patrón Fachada resulta útil en estas situaciones. Este patrón proporciona una interfaz simplificada de alto nivel para un subsistema, reduciendo el acoplamiento y las dependencias. Esto, a su vez, facilita el uso del subsistema y lo hace más manejable.

Una fachada es una clase que ofrece esta interfaz simplificada para ser utilizada por los clientes. Con un objeto Fachada, los clientes interactúan con dicho objeto en lugar de interactuar directamente con las clases del subsistema. El objeto Fachada asume la responsabilidad de interactuar con los componentes del subsistema. En efecto, los clientes se comunican con la fachada para trabajar con el subsistema. El patrón Fachada promueve un acoplamiento débil entre el subsistema y sus clientes (Figura 2).

Como se puede observar en la Figura 2, el objeto Fachada desacopla a los clientes de los objetos del subsistema. Cuando una clase del subsistema sufre un cambio, los clientes ya no se ven afectados como antes.

Aunque los clientes utilizan la interfaz simplificada proporcionada por la fachada, cuando sea necesario, un cliente podrá acceder directamente a los componentes del subsistema a través de sus interfaces de nivel más bajo, como si el objeto Fachada no existiera. En ese caso, seguirán teniendo los mismos problemas de dependencia y acoplamiento que antes.

Figura 1 — Interacción del Cliente con las Clases del Subsistema antes de Aplicar el Patrón Fachada

[Cliente A, Cliente B] → [Clases del Subsistema] (acoplamiento directo, alto)

Figura 2 — Interacción del Cliente con las Clases del Subsistema después de Aplicar el Patrón Fachada

[Cliente A, Cliente B] → [Fachada] → [Clases del Subsistema] (acoplamiento débil)

2. Ejemplo

Construyamos una aplicación que:

- Acepte los datos del cliente (información de cuenta, dirección y tarjeta de crédito).
- Valide los datos de entrada.
- Guarde los datos de entrada en los archivos de datos correspondientes.

Supongamos que existen tres clases — Cuenta, Dirección y TarjetaCredito — (Listado 1 al Listado 3) disponibles en el sistema, cada una con sus propios métodos para validar y guardar los datos respectivos (Figura 3).

Construyamos un cliente GestorCuenta (Listado 4) que muestre una interfaz de usuario para ingresar los datos del cliente. Al ejecutar el cliente GestorCuenta, este muestra la interfaz de usuario que se aprecia en la Figura 4.

Para validar y guardar los datos de entrada, el cliente GestorCuenta tendría que:

- Crear objetos Cuenta, Dirección y TarjetaCredito.
- Validar los datos de entrada usando estos objetos.
- Guardar los datos de entrada usando estos objetos.

El diagrama de secuencia de la Figura 5 representa el flujo de mensajes entre estos objetos.

Aplicar el patrón Fachada en este caso puede conducir a un mejor diseño, ya que promueve un bajo acoplamiento entre el cliente y las clases del subsistema (las clases Dirección, Cuenta y TarjetaCredito en este caso).

Aplicando el patrón Fachada, definimos una clase Fachada ClienteFachada (Figura 6 y Listado 5) que ofrece una interfaz simplificada de alto nivel para el subsistema compuesto por las clases de procesamiento de datos del cliente (Dirección, Cuenta y TarjetaCredito).

La clase ClienteFachada ofrece un servicio de negocio de alto nivel en la forma del método guardarDatosCliente. En lugar de interactuar con los componentes del subsistema directamente, el cliente GestorCuenta puede hacer uso de la interfaz de alto nivel y más simplificada que ofrece el objeto ClienteFachada para validar y guardar los datos del cliente de entrada (Figura 7).

En el diseño revisado, para validar y guardar los datos del cliente de entrada, el cliente necesita:

- Crear u obtener una instancia de la fachada ClienteFachada.
- Enviar los datos a validar y guardar a la ClienteFachada mediante sus métodos setter.
- Invocar el método guardarDatosCliente sobre la instancia de ClienteFachada.

La ClienteFachada se encarga de los detalles de creación de los objetos necesarios del subsistema y de invocar los métodos apropiados sobre dichos objetos para validar y guardar los datos del cliente. El cliente ya no necesita acceder directamente a los objetos del subsistema (Cuenta/Dirección/TarjetaCredito).

La Figura 8 muestra el flujo de mensajes en el diseño revisado.

Listado 1 — Clase Cuenta

```
public class Cuenta {  
    String nombre;  
    String apellido;  
    final String ARCHIVO_CUENTA = "DatosCuenta.txt";
```

```

public Cuenta(String nombre, String apellido) {
    this.nombre = nombre;
    this.apellido = apellido;
}

public boolean esValida() {
    /*
     * Validación simplificada para mantener
     * el ejemplo más claro.
     */
    ...
}

public boolean guardar() {
    UtilArchivo util = new UtilArchivo();
    String linea = getApellido() + "," + getNombre();
    return util.escribirEnArchivo(ARCHIVO_CUENTA, linea, true, true);
}

public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
}

```

Listado 2 — Clase Dirección

```

public class Direccion {
    String direccion;
    String ciudad;
    String estado;
    final String ARCHIVO_DIRECCION = "Direccion.txt";

    public Direccion(String dir, String ciu, String est) {
        direccion = dir;
        ciudad = ciu;
        estado = est;
    }

    public boolean esValida() {
        /*
         * El algoritmo de validación de direcciones
         * puede ser complejo en aplicaciones reales.
         * Se utiliza validación simplificada.
         */
        if (getEstado().trim().length() < 2) return false;
        return true;
    }

    public boolean guardar() {
        UtilArchivo util = new UtilArchivo();
        String linea = getDireccion() + "," + getCiudad() + "," + getEstado();
        return util.escribirEnArchivo(ARCHIVO_DIRECCION, linea, true, true);
    }

    public String getDireccion() { return direccion; }
    public String getCiudad() { return ciudad; }
    public String getEstado() { return estado; }
}

```

Listado 3 — Clase TarjetaCredito

```

public class TarjetaCredito {
    String tipoTarjeta;
    String numeroTarjeta;
}

```

```

String fechaVencimiento;
final String ARCHIVO_TC = "TC.txt";

public TarjetaCredito(String tipo, String numero, String fechaVenc) {
    tipoTarjeta      = tipo;
    numeroTarjeta     = numero;
    fechaVencimiento  = fechaVenc;
}

public boolean esValida() {
    /*
     * Validación simplificada para mantener
     * el ejemplo más claro.
     */
    if (getTipo().equals(GestorCuenta.VISA))
        return (getNumero().trim().length() == 16);
    if (getTipo().equals(GestorCuenta.DISCOVER))
        return (getNumero().trim().length() == 15);
    if (getTipo().equals(GestorCuenta.MASTER))
        return (getNumero().trim().length() == 16);
    return false;
}

public boolean guardar() {
    UtilArchivo util = new UtilArchivo();
    String linea = getTipo() + "," + getNumero() + "," + getFechaVenc();
    return util.escribirEnArchivo(ARCHIVO_TC, linea, true, true);
}

public String getTipo()      { return tipoTarjeta; }
public String getNumero()    { return numeroTarjeta; }
public String getFechaVenc() { return fechaVencimiento; }
}

```

Figura 3 — Clases del Subsistema para Validar y Guardar los Datos del Cliente

Cuenta	Direccion	TarjetaCredito
<i>esValida():boolean</i> <i>guardar():boolean</i> <i>getNombre():String</i> <i>getApellido():String</i> <i>nombre:String</i> <i>apellido:String</i>	<i>esValida():boolean</i> <i>guardar():boolean</i> <i>getDireccion():String</i> <i>getEstado():String</i> <i>direccion:String</i> <i>ciudad:String</i> <i>estado:String</i>	<i>esValida():boolean</i> <i>guardar():boolean</i> <i>getTipo():String</i> <i>getNumero():String</i> <i>getFechaVenc():String</i> <i>tipoTarjeta:String</i> <i>numeroTarjeta:String</i> <i>fechaVencimiento:String</i>

Listado 4 — Clase Cliente GestorCuenta

```

public class GestorCuenta extends JFrame {
    public static final String NUEVA_LINEA    = "\n";
    public static final String VALIDAR_GUARDAR = "Validar & Guardar";
    public static final String VISA           = "Visa";
    public static final String MASTER         = "MasterCard";
    public static final String DISCOVER       = "Discover";
    ...

    public GestorCuenta() {
        super(" Patrón Fachada - Ejemplo ");
        cmbTipoTarjeta = new JComboBox();
        cmbTipoTarjeta.addItem(GestorCuenta.VISA);
        cmbTipoTarjeta.addItem(GestorCuenta.MASTER);
        cmbTipoTarjeta.addItem(GestorCuenta.DISCOVER);
    }
}

```

```

        ...
        JButton btnValidarGuardar =
            new JButton(GestorCuenta.VALIDAR_GUARDAR);
        ...
    }

    public String getNombre() {
        return txtNombre.getText();
    }
    ...
}

```

Figura 4 — Interfaz de Usuario para Ingresar los Datos del Cliente

[Campos: Nombre, Apellido, Dirección, Ciudad, Estado, Tipo Tarjeta (Visa ▼), Número Tarjeta, Fecha Venc. — Botones: Validar & Guardar | Salir]

Figura 5 — Flujo de Mensajes en el Diseño Original (Interacción Directa del Cliente con el Subsistema)

:GestorCuenta → Cuenta(nombre, apellido) → esValida() → Direccion(dir, ciudad, estado) → esValida() → TarjetaCredito(tipo, numero, fechaVenc) → esValida() → guardar() × 3

Figura 6 — Clase ClienteFachada utilizada por el Cliente en el Diseño Revisado

ClienteFachada

```

setDireccion(inDireccion:String)
setCiudad(inCiudad:String)
setEstado(inEstado:String)
setTipoTarjeta(inTipo:String)
setNumeroTarjeta(inNumero:String)
setFechaVenc(inFechaVenc:String)
setNombre(inNombre:String)
setApellido(inApellido:String)
guardarDatosCliente():boolean

```

```

direccion:String
ciudad:String
estado:String
tipoTarjeta:String
numeroTarjeta:String
fechaVencimiento:String
nombre:String
apellido:String

```

Listado 5 — Clase ClienteFachada

```

public class ClienteFachada {
    private String direccion;
    private String ciudad;
    private String estado;
    private String tipoTarjeta;
    private String numeroTarjeta;
    private String fechaVencimiento;
    private String nombre;
    private String apellido;

    public void setDireccion(String inDireccion) { direccion = inDireccion; }
    public void setCiudad(String inCiudad) { ciudad = inCiudad; }
    public void setEstado(String inEstado) { estado = inEstado; }
    public void setNombre(String inNombre) { nombre = inNombre; }
}

```

```

    public void setApellido(String inApellido)      { apellido      = inApellido;
}
    public void setTipoTarjeta(String inTipo)      { tipoTarjeta   = inTipo; }
    public void setNumeroTarjeta(String inNumero) { numeroTarjeta = inNumero; }
    public void setFechaVenc(String inFechaVenc)   { fechaVencimiento =
inFechaVenc; }

    public boolean guardarDatosCliente() {
        Direccion      objDireccion;
        Cuenta          objCuenta;
        TarjetaCredito objTarjeta;

        /*
         * Las siguientes operaciones relacionadas con el subsistema
         * son transparentes para el cliente.
         */
        boolean datosValidos = true;
        String mensajeError = "";

        objCuenta = new Cuenta(nombre, apellido);
        if (!objCuenta.esValida()) {
            datosValidos = false;
            mensajeError = "Nombre/Apellido no válidos";
        }

        objDireccion = new Direccion(direccion, ciudad, estado);
        if (!objDireccion.esValida()) {
            datosValidos = false;
            mensajeError = "Dirección/Ciudad/Estado no válidos";
        }

        objTarjeta = new TarjetaCredito(tipoTarjeta, numeroTarjeta,
fechaVencimiento);
        if (!objTarjeta.esValida()) {
            datosValidos = false;
            mensajeError = "Información de Tarjeta no válida";
        }

        if (!datosValidos) {
            System.out.println(mensajeError);
            return false;
        }

        if (objDireccion.guardar() && objCuenta.guardar() && objTarjeta.guardar())
        {
            return true;
        } else {
            return false;
        }
    }
}

```

Figura 7 — Asociación de Clases con la Fachada en el Diseño Revisado

GestorCuenta → ClienteFachada → { Cuenta, Dirección, TarjetaCredito }

Figura 8 — En el Diseño Revisado, el Cliente Interactúa con la Fachada en lugar del Subsistema

:GestorCuenta → guardarDatosCliente() → :ClienteFachada → Cuenta(nombre, apellido) → esValida() → Direccion(...) → esValida() → TarjetaCredito(...) → esValida() → guardar() × 3

3. Notas Importantes

A continuación se presentan algunas consideraciones a tener en cuenta al aplicar el patrón Fachada:

- Una fachada no debe diseñarse para añadir funcionalidad adicional a las clases del subsistema.
-

```
TarjetaCredito getTarjetaCredito()
```

expone el subsistema a los clientes, y la aplicación podría no aprovechar todos los beneficios del patrón Fachada.

- El objetivo de una fachada es proporcionar una interfaz de alto nivel; por ello, preferiblemente un método típico de Fachada debe ofrecer un servicio de negocio de alto nivel en lugar de realizar tareas individuales de bajo nivel.

4. Preguntas de Práctica

1. Diseñe e implemente una fachada que pueda ser utilizada por diferentes objetos cliente para crear una solicitud de compra compuesta por distintas líneas de artículos, datos de encabezado y otra información.
2. Mejore la misma clase Fachada para ofrecer métodos de servicios de negocio adicionales con el fin de:
 - Recuperar una solicitud de compra de una base de datos.
 - Crear una nueva solicitud de compra copiando una solicitud de compra existente.